CSC413/CSE315

Design of Operating System

Programming Assignment – Summer 2025

Due on: 31th August 2025

Please read the following information very carefully.

- The assignment is divided into 2 sections: a **mandatory** section and an **optional** section. The mandatory section is worth 100 marks. *The optional section is not mandatory!!!!*
- Ensure you submit on time. Late submissions may be penalized.
- **Answer** every question to the **best of your abilities** and **do not resort to plagiarism** as you will be asked to present your answers at **viva**. Failure to show competence according to your answers will be marked as plagiarism and dealt with accordingly.
- The optional section contains difficult and tough questions which most may not be able to answer properly. **Only attempt these questions after you are done with the mandatory set.** That being said, successfully answering any optional question will merit you much praise and significant consideration in the grading for this course.
- Submissions are individual. *However, you may participate in a group to solve the optional section problems. Clearly state all students who worked in the group to merit this bonus.*
- Attempt to **answer every question in** the **mandatory** section in its own code file/folder.
- Ensure your deliverables are in a **zip** file with the following prefix.
  `"<Section Number>_<ID>_<Course Name>_<Student Name>.zip"`
  ex: 3_2120117_CSC413-Ezhar Uddin.zip
  ex: 1_2091012_CSE315_Rezwan_Ahmed.zip
  This helps with automation. Your submission may not be graded if this prefix does not match.
- **WARNING: No code submission will be accepted inside a PDF or Word document**. Ensure all your code exists as code/project files that can be easily executed/run on demand.
- If needed, any textual justifications or descriptions of how you solved a problem can be accepted, either through code comments or if necessary, a separate PDF or Word document. But the questions will be asked primarily in viva.
- For **viva**, ensure you have a **laptop** that is at the very least, capable of running all the applications in the mandatory section. You may need the following:
  - An IDE capable of compiling Java 8/JavaFX [ex: IntelliJ/NetBeans]
  - Linux Desktop Environment + Build tools [ex: Ubuntu/Red Hat]
  - Windows Desktop Environment + An IDE capable of compiling WIN32 applications [ex: CodeBlocks]

  If you do not have this, you do not need to appear for the viva.
- All students who worked in a group to solve an optional problem must be present at the viva together.
- You have to appear at the viva only for the questions which you have answered successfully. If you did not submit the programming assignment you do not need to appear for the viva.

Mandatory Section

YOU SHOULD ANSWER EVERY QUESTION IN THIS SECTION. SUBMIT THE CODE/PROJECTS VIA ZIP SUBMISSION AND PREPARE TALKING POINTS FOR THE QUESTIONS IN THE BULLETED LISTS.

A. **Multiprocessing:** Write a Linux multiprocessing program. **[20]**

The Collatz conjecture concerns what happens when we take any positive integer n and apply the following algorithm:

$$n = \begin{cases} n/2, \text{ if } n \text{ is even} \\ 3 \times n + 1, \text{ if } n \text{ is odd} \end{cases}$$

The conjecture states that when this algorithm is continually applied, all positive integers will eventually reach 1. For example, if n = 35, the sequence is

35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1

Write a C program using the `fork()` system call that generates this sequence in the child process. The starting number will be provided from the command line. For example, if 8 is passed as a parameter on the command line, the child process will output 8, 4, 2, 1. Because the parent and child processes have their own copies of the data, it will be necessary for the child to output the sequence. Have the parent invoke the `wait()` call to wait for the child process to complete before exiting the program. Perform necessary error checking to ensure that a positive integer is passed on the command line.

Your job is to:

- Explain how you solved the problem.
- Perform the required task successfully using multiprocessing.
- Produce valid output every time the program is run.
- Be capable of handling various kinds of input.

B. **CPU Scheduling:** Write a CPU scheduler simulator. **[20]**

A multi-level CPU scheduler simulation in an OS project involves creating a system that manages processes using multiple queues, each with its own scheduling algorithm. This allows for different types of processes to be handled with optimized strategies. The simulation should demonstrate how processes move between queues, how scheduling algorithms (like Round Robin or Priority Scheduling) are applied within each queue, and how this affects overall system performance metrics.

Key aspects you should consider for the simulation:

I. Multilevel Queue Structure:
   - Define multiple queues, each representing a different priority level or process type.
   - For example, one queue might be for real-time processes, another for interactive processes, and another for batch jobs.

II. Queue Scheduling Algorithms:
   - Each queue can have its own scheduling algorithm, such as Round Robin, Priority Scheduling, or FIFO.
   - Implement these algorithms within the simulation.

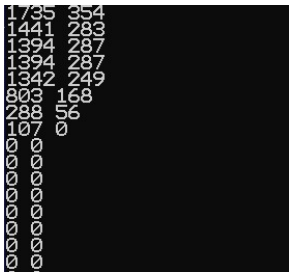
III. Process Arrival and Movement:
    - Simulate processes arriving with different characteristics (e.g., arrival time, burst time, priority).

IV. Performance Metrics: Track key performance indicators like:
    - CPU Utilization: The percentage of time the CPU is busy.
    - Throughput: The number of processes completed per unit of time.
    - Turnaround Time: The time it takes for a process to complete from arrival to finish.
    - Waiting Time: The time a process spends waiting in the ready queue.
    - Response Time: The time it takes for a process to produce its first output.

V. Preemptive vs. Non-Preemptive:
    - Decide whether the simulation will be preemptive (allowing a higher-priority process to interrupt a running process) or non-preemptive.

VI. Implementation Language:
    - Choose a suitable language like Java, Python, or C/C++ to implement the simulation.

C. WIN32 Programming: Write a small C/C++ Windows application that utilizes the WIN32 API. **[20]**
The program is a simple command-line Windows executable that constantly prints the position of the mouse cursor, in terms of screen coordinates x and y. You are free to dictate the rate at which the cursor position updates but ensure that the mouse position continues to update even when the application loses focus. You must only use the WIN32 API to obtain this information. Refer to the picture for understanding.



The program must be a native Windows C/C++ application: using other libraries to obtain the mouse position like JavaFX, SDL or GTK is not allowed. You should also be able to describe:

- How you installed and used the Windows SDK
- How you solved the implementation
- Documentation/resources you needed to refer to

D. **Multithreading:** Write a C/C++ application that employs multiple threads to produce a Mandelbrot Set image **[20]**

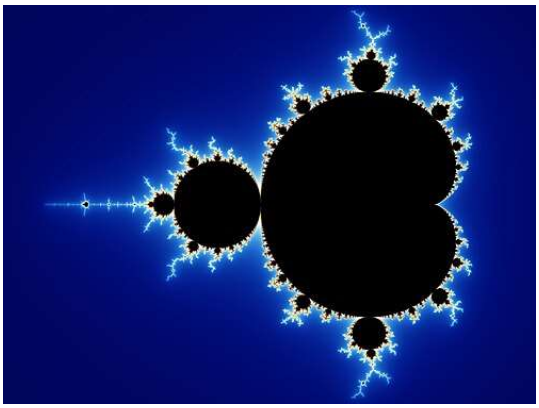
The program should generate a PPM image of a portion of the Mandelbrot set, a very well-known fractal. This should be done by using multiple threads to compute the image in parallel.

Mandelbrot Set Generation Algorithm: The Mandelbrot Set is defined as the set of all complex numbers c such that the recurrence relation

$$z_{i+1} = z_i^2 + c$$

remains bounded (i.e. the absolute value of $|z_i|$ does not diverge to infinity) when starting with $z_i = 0$.

For a given point (x, y) in the complex plane, we iterate the equation above and check whether $|z_n|$ exceeds a certain threshold (commonly 2) within a set number of iterations. If it does, the point is outside the set; otherwise, it is inside. The number of iterations before divergence is used to determine the pixel's color. For the sake of simplicity, you can consider grayscale where brighter means fewer iterations and darker means more iterations, but traditional Mandelbrot set images are in full color.



The Mandelbrot set within the region $(-2 - 1.12i)$ to $(0.47 + 1.12i)$. All images generated should be a subset inside this region as the Mandelbrot Set does have any peculiar features outside this range.

The program should divide the given coordinate space it receives into rows/columns of pixels, then spawn 8 threads, each responsible for computing the pixel values for a subset of the rows. Each pixel's color is determined by mapping its coordinates to the complex plane and performing the Mandelbrot iteration. Once all threads complete, the main thread should gather the results into an image file format called PPM(an extremely simple text-based image format) and save it to disk.

You can view your PPM images using this site.

The arguments to the program should look like this

```
./mandelbrot <img_width> <img_height> <xmin> <xmax> <ymin> <ymax> <max_iterations>
<output_file>
```

As an example, we should be able to produce the image above like this

```
./mandelbrot 800 600 -2.0 0.47 -1.12 1.12 1000 mandelbrot.ppm
```

Your job is to:

- Explain how you solved the problem.
- Perform the required task successfully using multithreading.
- Produce a valid, consistent image every time the program is run.
- Be capable of handling various resolutions and coordinate ranges.
- Be capable of correctly rendering different zoom levels and iteration depths.

E. **Synchronization:** Implement a synchronization paradigm. **[20]**

The Dining Philosophers problem is a classic synchronization problem in operating systems. It involves multiple processes (philosophers) competing for limited resources (chopsticks) needed to perform a task (eating). The goal is to simulate this problem using threads and synchronization mechanisms (like semaphores or mutexes) to prevent deadlock and starvation

Consider the breakdown of the key aspects and steps involved in simulating the Dining Philosophers problem:

I. Problem Definition:

- Five philosophers sit at a circular table, each with a plate of food and a chopstick on either side.
- Philosophers alternate between thinking and eating.
- To eat, a philosopher needs to pick up both chopsticks adjacent to their plate.
- If a philosopher picks up only one chopstick, they must wait for the other to become available.
- The problem arises when all philosophers try to pick up their left (or right) chopstick simultaneously, leading to a deadlock where no one can eat.

II. Simulation Setup:

- **Threads:** Use threads to represent each philosopher.
- **Resources:** Represent chopsticks as shared resources (e.g., using mutexes or semaphores).
- **States:** Define states for each philosopher: thinking, hungry, and eating.
- **Synchronization:** Implement synchronization mechanisms to control access to the chopsticks and prevent deadlocks and starvation.

III. Implementing the Simulation:

- **Initialization:**
  - Create threads for each philosopher.
  - Initialize the chopsticks as available (e.g., using semaphores initialized to 1, or mutexes).
- **Philosopher Logic:**
  - Each philosopher:
    - First, enters the "thinking" state.
    - After a random time, enters the "hungry" state.
    - Attempts to acquire both chopsticks (using mutexes or semaphores):
      - If both chopsticks are available, the philosopher grabs them and enters the "eating" state.
      - If one or both chopsticks are unavailable, the philosopher waits until they become available.
    - Eats for a random amount of time.
    - Releases both chopsticks.
    - Returns to the "thinking" state.
- **Deadlock Prevention:**
  - **Asymmetric Approach:** Have some philosophers pick up the left chopstick first and others pick up the right chopstick first.
  - **Timeout:** Implement a timeout mechanism where a philosopher releases chopsticks if they can't acquire both within a certain time.

IV. Evaluating the Simulation:

- **Deadlock Detection:** Observe if the simulation enters a deadlock state where no philosopher can proceed.
- **Starvation Detection:** Observe if any philosopher is repeatedly denied access to chopsticks, leading to starvation.

Hint: Java (implementation)

When a philosopher wishes to eat, she invokes the method `takeForks(philosopherNumber)`, where `philosopherNumber` identifies the number of the philosopher wishing to eat. When a philosopher finishes eating, she invokes returnForks(philosopherNumber).

Your solution should implement the following interface:

```
public interface DiningServer
{
    /* Called by a philosopher when it wishes to eat */
    public void takeForks(int philosopherNumber);
    /* Called by a philosopher when it is finished eating */
    public void returnForks(int philosopherNumber);
}
```

It will require the use of Java condition variables

Optional Section

ONLY ATTEMPT THIS SECTION IF YOU HAVE COMPLETED THE MANDATORY SECTION. ANSWERING ANY ONE PROBLEM IS SUFFICIENT BUT FEEL FREE TO SOLVE MORE IF YOU ARE CAPABLE. YOU CAN USE ANY AND EVERY RESOURCE AVAILABLE TO YOU TO SOLVE THESE PROBLEMS, SHORT OF PLAGIARISM.

A. **Introduction to Linux Kernel Programming:** In this project, you will learn how to create a kernel module and load it into the Linux kernel.

This project will require the use of a Linux OS and possibly the source code of the kernel you are using. Although you may use any text editor to write these C programs, you will have to use the terminal application to compile the programs, and you will have to enter commands on the command line to manage the modules in the kernel.

As you'll discover, the advantage of developing kernel modules is that it is a relatively easy method of interacting with the kernel, thus allowing you to write programs that directly invoke kernel functions. It is important for you to keep in mind that you are indeed writing kernel code that directly interacts with the kernel. That normally means that any errors in the code could crash the system!

I. Kernel Modules Overview

The first part of this project involves following a series of steps for creating and inserting a module into the Linux kernel. You can list all kernel modules that are currently loaded by entering the command

```
$ lsmod
```

This command will list the current kernel modules in three columns: name, size, and where the module is being used.

```c
/* simple.c: a file that demonstrates Linux modules */
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>

/* This function is called when the module is loaded. */
int simple_init(void)
{
    printk(KERN_INFO "Loading Kernel Module\n"); return 0;
}

/* This function is called when the module is removed. */
void simple_exit(void)
{
    printk(KERN_INFO "Removing Kernel Module\n");
}

/* Macros for registering module entry and exit points. */

module_init(simple_init);
module_exit(simple_exit);
MODULE_LICENSE("GPL");

MODULE_DESCRIPTION("Simple Module");
MODULE_AUTHOR("SGG");
```

The program above is a very basic kernel module that prints appropriate messages when it is loaded and unloaded. The function `simple_init()` is the module entry point, which represents the function that is invoked when the module is loaded into the kernel. Similarly, the `simple_exit()` function is the module exit point— the function that is called when the module is removed from the kernel.

The module entry point function must return an integer value, with 0 representing success and any other value representing failure. The module exit point function returns void. Neither the module entry

point nor the module exit point are passed any parameters. The two following macros are used for registering the module entry and exit points with the kernel:

```
module init(simple_init);
module exit(simple_exit);
```

Notice in the figure how the module entry and exit point functions make calls to the `printk()` function. `printk()` is the kernel equivalent of `printf()`, but its output is sent to a kernel log buffer whose contents can be read by the `dmesg` command. One difference between `printf()` and `printk()` is that `printk()` allows us to specify a priority flag, whose values are given in the `<linux/printk.h>` include file. In this instance, the priority is `KERN_INFO`, which is defined as an informational message.

The final lines — `MODULE_LICENSE()`, `MODULE_DESCRIPTION()`, and `MODULE_AUTHOR()` — represent details regarding the software license, description of the module, and author. For our purposes, we do not require this information, but we include it because it is standard practice in developing kernel modules and compilation will fail without a valid license.

This kernel module *simple.c* is compiled using the Makefile accompanying the source code with this project. To compile the module, enter the following on the command line:

```
$ make
```

The compilation produces several files. The file *simple.ko* represents the compiled kernel module. The following step illustrates inserting this module into the Linux kernel.

II. Loading and Removing Kernel Modules

Kernel modules are loaded using the `insmod` command, which is run as follows:

```
$ sudo insmod simple.ko
```

To check whether the module has loaded, enter the `lsmod` command and search for the module *simple*. Recall that the module entry point is invoked when the module is inserted into the kernel. To check the contents of this message in the kernel log buffer, enter the command

```
$ sudo dmesg
```

You should see the message "Loading Module." Removing the kernel module involves invoking the `rmmod` command (notice that the '.ko' suffix is unnecessary):

```
$ sudo rmmod simple
```

Be sure to check with the `dmesg` command to ensure the module has been removed. Because the kernel log buffer can fill up quickly, it often makes sense to clear the buffer periodically. This can be accomplished as follows:

```
$ sudo dmesg -c
```

Proceed through the steps described above to create the kernel module and to load and unload the module. Be sure to check the contents of the kernel log buffer using `dmesg` to ensure that you have followed the steps properly.

As kernel modules are running within the kernel, it is possible to obtain values and call functions that are available only in the kernel and not to regular user applications. For example, the Linux include file `<linux/hash.h>` defines several hashing functions for use within the kernel. This file also defines the constant value `GOLDEN_RATIO_PRIME` (which is defined as an `unsigned long`).

This value can be printed out as follows:

```
printk(KERN_INFO "%lu\n", GOLDEN_RATIO_PRIME);
```

As another example, the include file `<linux/gcd.h>` defines the following function.

```
unsigned long gcd(unsigned long a, unsigned b);
```

which returns the greatest common divisor of the parameters a and b.

III. Submission

Once you can correctly load and unload a module, write a kernel module to complete the following additional steps:

- Print out the value of `GOLDEN_RATIO_PRIME` in the `module_init()` function.
- Print out the greatest common divisor of 3,700 and 24 in the `module_exit()` function.

You are expected to be able to demonstrate the compilation, explain problems/challenges you faced, and show successful loading/unloading of the module on your system.

B. **CSE 213/CSC305 Project Extension:** In your previous study of Object-Oriented Programming, you were asked to develop a real-world Java GUI application that sought to solve some IT/digital needs of some business or organization. Modify that project so it is capable of RMI and client-server communications.

Your task is to investigate and point out aspects in your project that could be serviced by some remote server instead of being implemented on the client.

Having a method implemented on the client makes it vulnerable to being exposed by reverse engineering or hacking: hence sensitive functions or functions that work with sensitive data are often implemented through remote APIs. Alternatively, a method may also be extremely computationally intensive, to the point where it is unfeasible to have it be processed client-side: such methods are also qualified to be RMI, and for such applications, the server is equipped with a lot of processing power to handle such calls on a large volume.

I. Submission

Once you have documented such use cases, implement working RMI or RPC systems to service some key methods. A significant amount of work must be done for this optional question to be considered completed. You are also required to explain

- What functions/actions have been implemented via remote execution methods
- Justifications for implementing said functions as RMI
- Demonstrate execution of RMI

II. Why this is necessary

Through revisiting past projects, students can identify areas for improvement, address any shortcomings, and implement new techniques or technologies that they may have learned since the project's completion. Moreover, extending previous projects provides an opportunity for students to apply their knowledge in a practical context, reinforcing key concepts and fostering a deeper appreciation for the subject matter.

Working incrementally on your projects also allows you to showcase your growth and development over time to future interviewers in job interviews and gives you more prominence in your career as a developer. As a result, we feel it is beneficial if you complete this optional and strongly recommend you take it seriously.

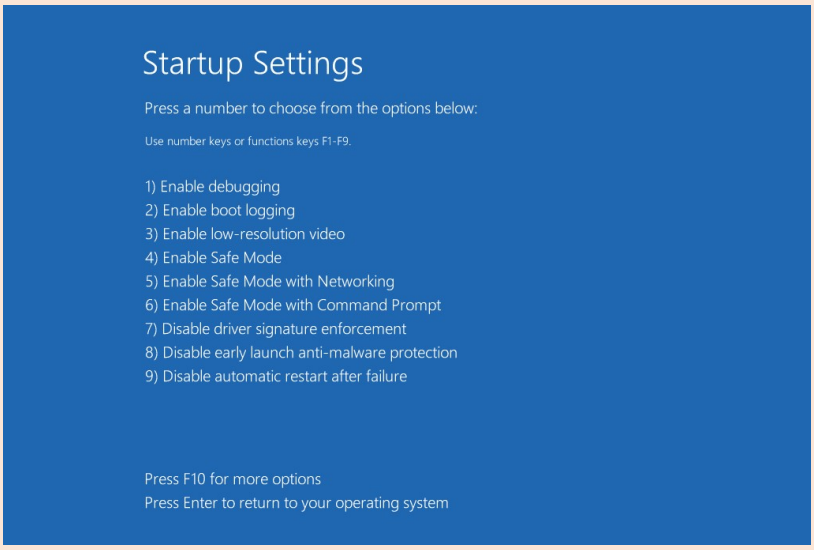
C. **WIN32 Driver Programming**: Demonstrate your ability to handle desktop applications programming and comprehend API and their documentation by writing a simple Windows driver.

Keep in mind WIN32 drivers are very similar to how Linux kernel modules work. No support will be provided for programming and compiling the driver: you must discover and find resources on the Internet that will help you to program a driver on your own. Once you have managed to do that,

I. Running the Driver:

The driver you have compiled should be something like `driver.sys`. However, getting any arbitrary driver to run on Windows is quite difficult compared to on Linux. This is because Windows employs **driver signing** to ensure that unsafe or dangerous code in drivers cannot be freely executed. The kernel-mode code signing policy enforces that kernel-mode drivers must be signed for the driver to load. Getting your drivers signed is only possible if you are a large enough company which can enter into an agreement with Microsoft or go through a strenuous approval process and have your drivers be thoroughly tested by WHQL. As amateur and niche OS programmers we want to avoid this hassle.

To allow your unsigned driver to execute, you must first restart your PC into **Startup Settings/Advanced Boot Options** by holding Shift while clicking on Power > Restart, or pressing F8 when your computer is booting. Then press F7 or 7 to **disable driver signature enforcement**.

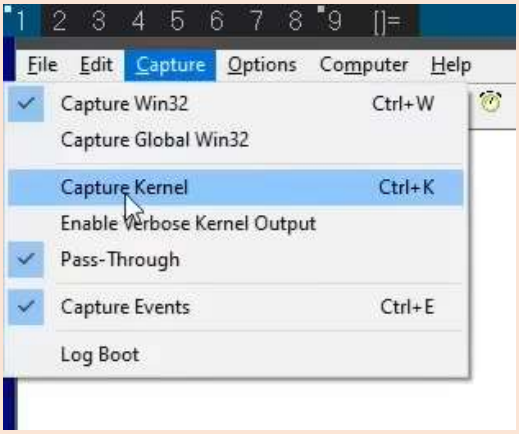


This allows unsigned drivers to temporarily execute during that session. After this, we need to create and start a service that uses our driver (this is somewhat like an entry-point for our driver, code in our driver needs to first start executing through some service). Open the command prompt with administrative privileges and type in

```
> sc
DESCRIPTION:
        SC is a command line program used for communicating with the
        Service Control Manager and services.
...
> sc create myDriver binPath= C:\Path\to\Driver\driver.sys type= kernel
[SC] CreateService SUCCESS
> sc start myDriver
The driver will execute without any prompt. Check DebugView for output.
> sc stop myDriver
This will unload the driver. Check DebugView for output (if you have handled it).
> sc delete myDriver
[SC] DeleteService SUCCESS
```

II. View output

Windows is an operating system primarily built for its customers, the masses. To provide an easy, seamless, and user-friendly experience for most people, there are very few built-in tools that allow programmers to tinker around and manipulate the OS. There is no built-in analogue of the Linux Kernel log viewer `dmesg`. Hence to view the output of the driver we will need an additional software called DebugView from Sysinternals. Run DebugView as an administrator before you start the driver in the section above.



Then click on **Capture > Capture Kernel** and **Enable Verbose Kernel Output** to ensure you can see the output produced by your driver.

III. Submission

Once you can correctly load and unload a driver, write a driver and complete the following additional steps:

- Print out "Hello Kernel! <your name>" using `DbgPrint()` in the driver entry function that appears only in the kernel log output captured by DebugView.
- Print out "Goodbye Kernel!" when your driver is unloaded that appears only in the kernel log output captured by DebugView.
- Prove your driver is executing code in kernel mode by initiating a crash: Use any suitable Kernel mode system call that will allow you to initiate a blue screen of death (BSOD). You are considered successful even if the system crashes before you being able to see the BSOD.

You are expected to be able to demonstrate the compilation, explain problems/challenges you faced, and show successful loading/unloading of the driver on your system.